Цель работы:

Освоить базовые объекты Kubernetes: Namespace, Pod, Deployment. Научиться описывать и развертывать приложения в кластере.

Теоретическая часть:

- **Namespace** — виртуальный кластер внутри физического.
- **Pod** — минимальная единица развертывания (1 или несколько контейнеров).
- **Deployment** — управляет жизненным циклом подов (обновления, масштабирование).

Создание пространства имен (namespace) в Kubernetes

Пространства имен в Kubernetes позволяют организовать ресурсы и управлять ими изолированно.

Запустите терминал. Убедитесь, что ваш kubectl настроен правильно и подключен к вашему кластеру Kubernetes. Вы можете проверить это с помощью команды:

```
kubectl cluster-info
```

```
C:\Windows\system32>kubectl cluster-info
Kubernetes control plane is running at https://kubernetes.docker.internal:6443
CoreDNS is running at https://kubernetes.docker.internal:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Если команда отобразит информацию о вашем кластере, вы подключены правильно.

Создайте пространство имен:

```
kubectl create namespace test
```

Проверьте созданные пространства имен:

Чтобы убедиться, что пространство имен было создано, выполните команду:

```
kubectl get namespaces
```

```
C:\Windows\system32>kubectl get namespace
NAME              STATUS   AGE
default           Active   26h
kube-node-lease   Active   26h
kube-public       Active   26h
kube-system       Active   26h
test              Active   25h
```

Вы должны увидеть ваше новое пространство имен в списке.

```
C:\Windows\system32>kubectl config set-context --current --namespace=test
Context "docker-desktop" modified.
```

```
C:\Windows\system32>kubectl get pod
No resources found in test namespace.
```

1. Pod

Модуль - группа контейнеров, которые представляют собой основной строительный блок к8с состоящий из одного или нескольких тесносвязанных контейнеров, которые будут выполняться вместе на 1 рабочем узле и 1 пространстве имен. Каждый модуль подобен отдельно логической машине с собственными процессами, ip. Все контейнеры работают только на 1 рабочем узле WorkNode).

Модуль при содержании нескольких контейнеров всегда работатт на 1 узле.

Далее проверим, какие PODы у нас вообще есть (все фразы будут подразумевать созданный `namespace - test`). В данном случае - никаких подов нет.

```
kubectl get pod
```

```
c:\Temp>kubectl get pod
No resources found in test namespace.
```

Рассмотрим несколько примеров POD:

Пример 1: создадим POD для `nginx`. Сперва создаем `pod1.yaml`, далее “активируем” файл, после чего проверяем созданный под. Через `describe` посмотрим характеристики пода. Там есть важная информация: как POD называется, в каком `namespace` находится, когда был создан, по какому образу, id контейнера, события и т.п.

spec - спецификация контейнера

Правила YAML для Kubernetes:

1. Отступы: Используйте пробелы (не табы)
2. Кодировка: UTF-8
3. Чувствительность к регистру: Все ключи чувствительны к регистру
4. Многострочные значения: Используйте | для сохранения переносов или > для их игнорирования
5. Комментарии: Начинаются с #
6. Типы данных: Строки можно указывать без кавычек, если нет специальных символов

```
C:\Windows\system32>cd c:\temp

c:\Temp>kubectl apply -f pod1.yaml
pod/hello created
```

```
c:\Temp>kubectl get pods
NAME     READY   STATUS    RESTARTS   AGE
hello    1/1     Running   0          45s
```

```
c:\Temp>kubectl describe pod hello
Name:             hello
Namespace:        test
Priority:         0
Service Account:  default
Node:             docker-desktop/192.168.65.3
Start Time:       Thu, 07 Nov 2024 12:20:47 +0300
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.1.0.14
IPs:
  IP:  10.1.0.14
Containers:
  container-hello:
    Container ID:   docker://5ab53648f66547e945aa70b109a618e7db120f1b042889668556fa285f150c90
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:28402db69fec7c17e179ea87882667f1e054391138f77ff
af0c3eb388efc3ffb
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Thu, 07 Nov 2024 12:20:53 +0300
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-h7frx (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
```

Чтобы проверить, что контейнер `nginx` работает, делаем `port-forward` с перенаправлением портов, например, на `5555` (можно взять любой). Далее пишем в браузере `localhost:5555` (в `minikube` вместо `localhost` возможно нужно написать `ip ноды` (введите `minikube ip`)) и проверяем. Все работает, все прекрасно. Удалим созданный под.

```
c:\Temp>kubectl port-forward hello 5555:80
Forwarding from 127.0.0.1:5555 -> 80
Forwarding from [::1]:5555 -> 80
Handling connection for 5555
Handling connection for 5555
```

localhost:5555

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

```
c:\Temp>kubectl get pods
NAME     READY   STATUS    RESTARTS   AGE
hello    1/1     Running   0          53m

c:\Temp>kubectl delete pod hello
pod "hello" deleted

c:\Temp>kubectl get pods
No resources found in test namespace.
```

Пример 2: Создадим POD с 2-я контейнерами: `nginx` и `tomcat`. Создадим `pod2.yaml`. Добавим `label` как некоторые метаданные. Через `describe` посмотрим характеристики подов. Потом опять перенаправим порты и посмотрим, что все работает

*pod2.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Pod
metadata:
  name: app-2
  labels:
    env: prod
    app: main
    tier: frontend
    owner: user
spec:
  containers:
  - name: container-hello
    image: nginx:latest
    ports:
      - containerPort: 80


  - name: container-tomcat
    image: tomcat:8.5.38
    ports:
      - containerPort: 8080
```

Стр 22, стлб 1 100% Windows (CRLF) UTF-8

```
c:\Temp>kubectl get pods
No resources found in test namespace.

c:\Temp>kubectl apply -f pod2.yaml
pod/app-2 created

c:\Temp>kubectl get pods
NAME     READY   STATUS    RESTARTS   AGE
app-2    2/2     Running   0          75s
```

```
c:\Temp>kubectl describe pod app-2
Name:             app-2
Namespace:        test
Priority:         0
Service Account:  default
Node:             docker-desktop/192.168.65.3
Start Time:       Thu, 07 Nov 2024 13:21:39 +0300
Labels:           app=main
                  env=prod
                  owner=user
                  tier=frontend
Annotations:      <none>
Status:           Running
IP:               10.1.0.16
IPs:
  IP:  10.1.0.16
Containers:
  container-hello:
    Container ID:   docker://a9a770ec0f4a48697a984da96bd68afe4d0d01969b9224668e328c3c16711573
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:28402db69fec7c17e179ea87882667f1e054391138f77ff
af0c3eb388efc3ffb
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Thu, 07 Nov 2024 13:21:45 +0300
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-4g5gw (ro)
  container-tomcat:
    Container ID:   docker://639057916a69e5bc2e5a43ee74ca553570c44c67e9b5124ea2d9a55cf04b2e4f
```

```
C:\Temp>kubectl port-forward app-2 5555:80
Forwarding from 127.0.0.1:5555 -> 80
Forwarding from [::1]:5555 -> 80

C:\Temp>kubectl port-forward app-2 5556:8080
Forwarding from 127.0.0.1:5556 -> 8080
Forwarding from [::1]:5556 -> 8080
Handling connection for 5556
Handling connection for 5556
```

localhost:5555

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Можно использовать одну команду для двух портов

```
C:\Temp>kubectl port-forward app-2 5555:80 5556:8080
Forwarding from 127.0.0.1:5555 -> 80
Forwarding from [::1]:5555 -> 80
Forwarding from 127.0.0.1:5556 -> 8080
Forwarding from [::1]:5556 -> 8080
```

Удаляем все

```
c:\Temp>kubectl delete pod app-2
pod "app-2" deleted

c:\Temp>kubectl get pods
No resources found in test namespace.
```

Задания для самостоятельного решения

Задание 3.1

Цель: закрепить навыки работы с командами описания и диагностики Pod'ов.

Задача:

Используя пример из лабораторной работы, создайте `Pod` с именем `explore-pod`, который использует образ `busybox:latest`.

В спецификации контейнера добавьте команду, которая будет поддерживать `Pod` в рабочем состоянии:

```
command: ["sh", "-c", "sleep 3600"]
```

Выполните следующие команды и проанализируйте их вывод:

```
kubectl get pod explore-pod -o wide
kubectl describe pod explore-pod
kubectl logs explore-pod
kubectl exec -it explore-pod -- cat /etc/os-release
```

Вопросы для отчета:

На каком узле (Node) запущен Pod?

Каков текущий статус Pod'а и контейнера?

Какой IP-адрес был назначен Pod'у?

Какая операционная система используется в контейнере busybox?

Задание 3.2

Цель: понять принципы взаимодействия контейнеров внутри одного Pod'а.

Задача:

Создайте `Pod debug-pod` с двумя контейнерами:

Основной контейнер: `nginx:alpine`

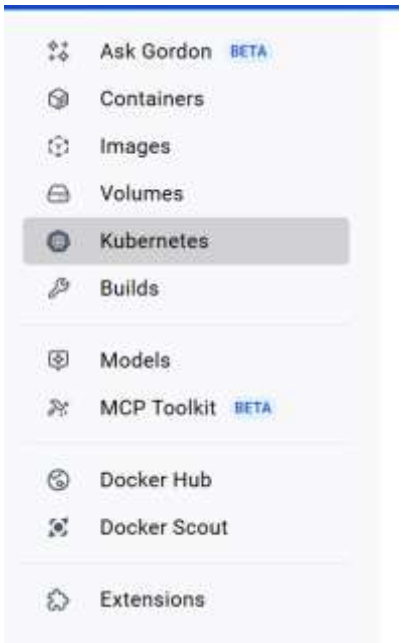
Вспомогательный контейнер: `busybox:latest` с командой `["sh", "-c", "while true; do echo $(date) >> /shared/log.txt; sleep 5; done"]`

Требования к YAML-манифесту:

- оба контейнера должны монтировать том в разные пути
- nginx должен работать на порту 80
- добавьте метки: app: debug, env: test

Дополнение

Все действия описанные а лабораторной работе можно выполнять и в Docker Desktop. Все команды пишутся в Terminal

Например,

Однако, кроме этого все существующие Pod можно увидеть и в соответствующем разделе